

# Deep Learning

Summative presentation : Individual Presentation for Machine Learning Module

# Problem Context, Objective and Approach

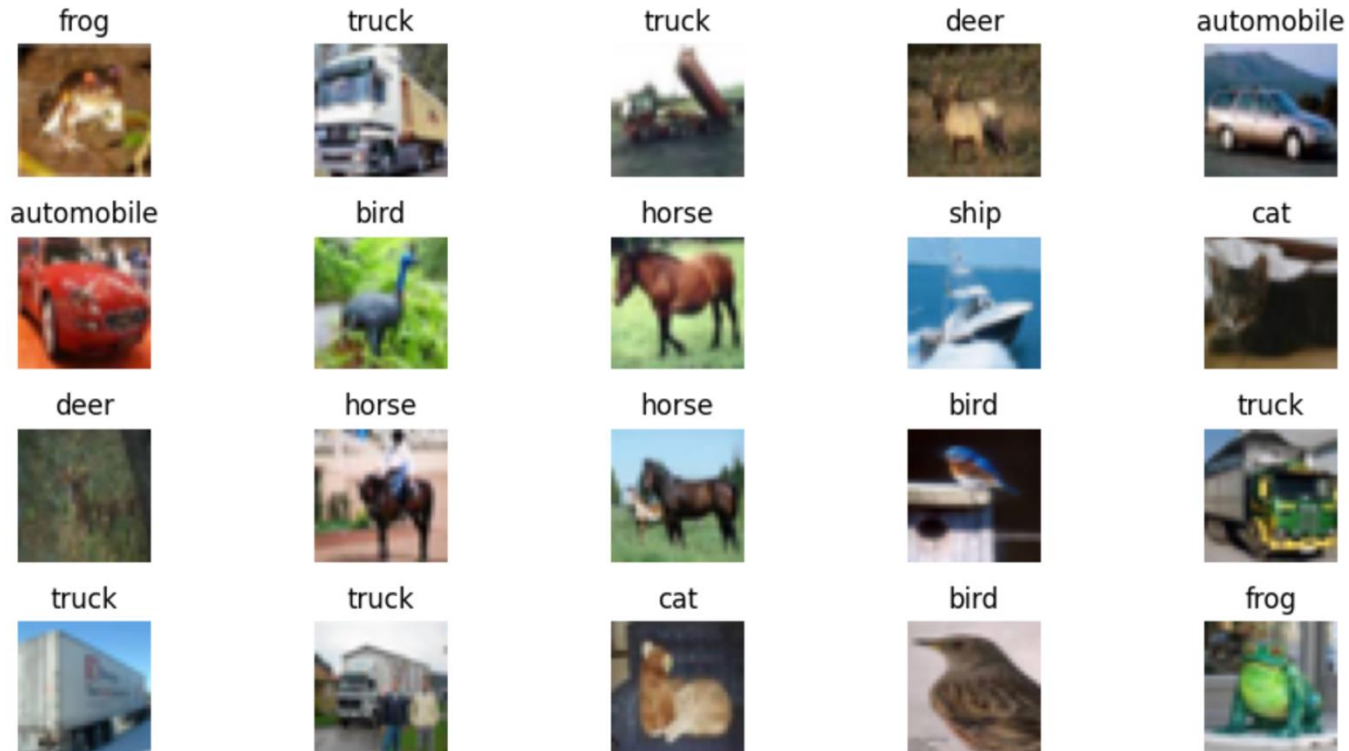
|            |                                                               |
|------------|---------------------------------------------------------------|
| Problem    | Object recognition classifies images into object categories   |
| Dataset    | CIFAR-10, 10 classes, 32×32 RGB images                        |
| Track      | Track 2: CNNs and Transfer Learning                           |
| Objective  | Build and evaluate CNN models for object recognition          |
| Evaluation | Accuracy, loss, precision, recall, F1-score, confusion matrix |

# CIFAR-10 Sample Images

```
# Display sample CIFAR-10 images
plt.figure(figsize=(10, 5))

for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(X_train_full[i])
    plt.title(class_names[int(y_train_full[i][0])])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



# Shape Output

The validation set was separated from the training data to monitor generalisation during model development. It helped identify overfitting, tune design choices such as dropout and data augmentation, and avoid using the test set until the final unbiased evaluation.

```
▶ from sklearn.model_selection import train_test_split

# Normalise pixel values from 0-255 to 0-1
X_train_full = X_train_full.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Split labels from shape (50000, 1) to (50000,)
y_train_full = y_train_full.flatten()
y_test = y_test.flatten()

# Create training and validation split
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full,
    y_train_full,
    test_size=10000,
    random_state=42,
    stratify=y_train_full
)

print("Training data:", X_train.shape)
print("Training labels:", y_train.shape)
print("Validation data:", X_val.shape)
print("Validation labels:", y_val.shape)
print("Test data:", X_test.shape)
print("Test labels:", y_test.shape)
```

```
... Training data: (40000, 32, 32, 3)
Training labels: (40000,)
Validation data: (10000, 32, 32, 3)
Validation labels: (10000,)
Test data: (10000, 32, 32, 3)
Test labels: (10000,)
```

# Split Table

```
import pandas as pd

split_summary = pd.DataFrame({
    "Dataset": ["Training", "Validation", "Test"],
    "Number of Images": [len(X_train), len(X_val), len(X_test)],
    "Purpose": [
        "Used to learn model weights",
        "Used to tune model design and detect overfitting",
        "Used only for final unbiased evaluation"
    ]
})

split_summary
```

| ... | Dataset    | Number of Images | Purpose                                          |
|-----|------------|------------------|--------------------------------------------------|
| 0   | Training   | 40000            | Used to learn model weights                      |
| 1   | Validation | 10000            | Used to tune model design and detect overfitting |
| 2   | Test       | 10000            | Used only for final unbiased evaluation          |

Next steps:

[Generate code with split\\_summary](#)

[New interactive sheet](#)

# Balance Table

```
import numpy as np
import pandas as pd

train_class_counts = pd.Series(y_train).value_counts().sort_index()
val_class_counts = pd.Series(y_val).value_counts().sort_index()
test_class_counts = pd.Series(y_test).value_counts().sort_index()

class_balance = pd.DataFrame({
    "Class": class_names,
    "Training": train_class_counts.values,
    "Validation": val_class_counts.values,
    "Test": test_class_counts.values
})

class_balance
```

...

|   | Class      | Training | Validation | Test |
|---|------------|----------|------------|------|
| 0 | airplane   | 4000     | 1000       | 1000 |
| 1 | automobile | 4000     | 1000       | 1000 |
| 2 | bird       | 4000     | 1000       | 1000 |
| 3 | cat        | 4000     | 1000       | 1000 |
| 4 | deer       | 4000     | 1000       | 1000 |
| 5 | dog        | 4000     | 1000       | 1000 |
| 6 | frog       | 4000     | 1000       | 1000 |
| 7 | horse      | 4000     | 1000       | 1000 |
| 8 | ship       | 4000     | 1000       | 1000 |
| 9 | truck      | 4000     | 1000       | 1000 |

Next steps:

[Generate code with class\\_balance](#)

[New interactive sheet](#)

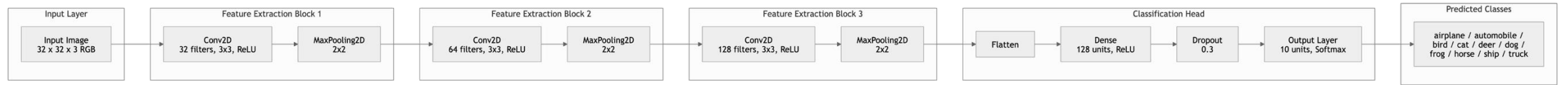
# Baseline CNN - Summary

... Model: "sequential\_3"

| Layer (type)                    | Output Shape       | Param # |
|---------------------------------|--------------------|---------|
| conv2d_9 (Conv2D)               | (None, 32, 32, 32) | 896     |
| max_pooling2d_9 (MaxPooling2D)  | (None, 16, 16, 32) | 0       |
| conv2d_10 (Conv2D)              | (None, 16, 16, 64) | 18,496  |
| max_pooling2d_10 (MaxPooling2D) | (None, 8, 8, 64)   | 0       |
| conv2d_11 (Conv2D)              | (None, 8, 8, 128)  | 73,856  |
| max_pooling2d_11 (MaxPooling2D) | (None, 4, 4, 128)  | 0       |
| flatten_3 (Flatten)             | (None, 2048)       | 0       |
| dense_6 (Dense)                 | (None, 128)        | 262,272 |
| dropout_3 (Dropout)             | (None, 128)        | 0       |
| dense_7 (Dense)                 | (None, 10)         | 1,290   |

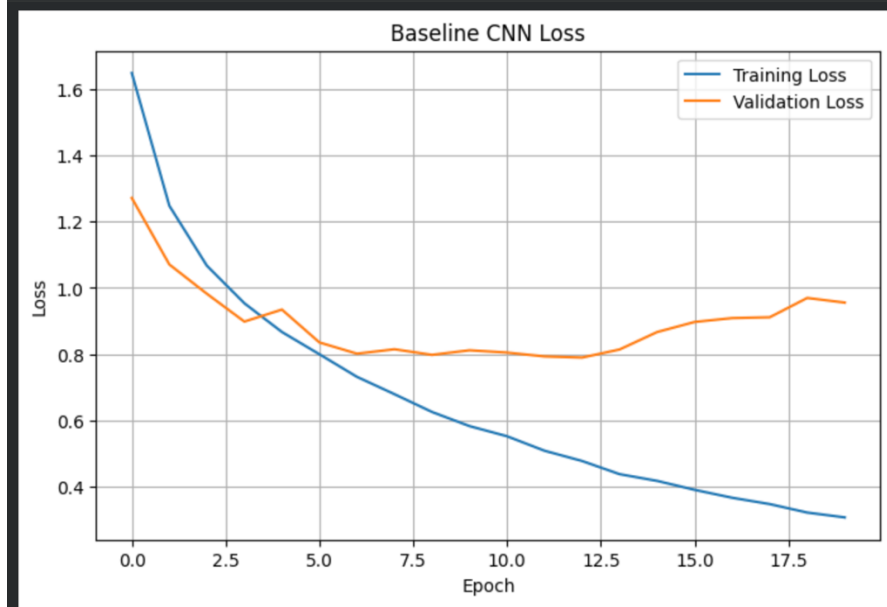
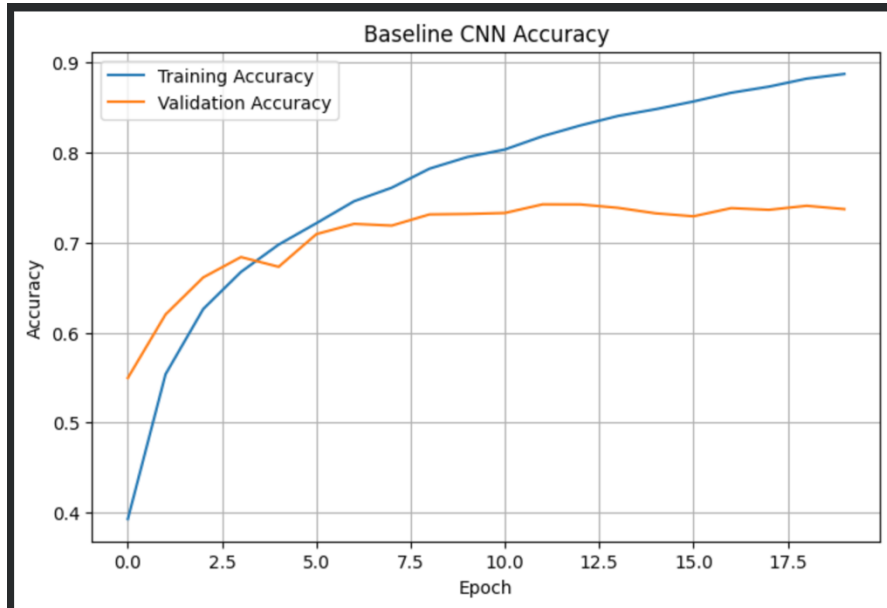
Total params: 356,810 (1.36 MB)  
Trainable params: 356,810 (1.36 MB)  
Non-trainable params: 0 (0.00 B)

# Baseline CNN - Architecture





# Baseline CNN - Results



| Metric                    | Result |
|---------------------------|--------|
| Final training accuracy   | 88.74% |
| Final validation accuracy | 73.72% |
| Test accuracy             | 74.26% |
| Final training loss       | 0.3078 |
| Final validation loss     | 0.9557 |
| Test loss                 | 0.9621 |

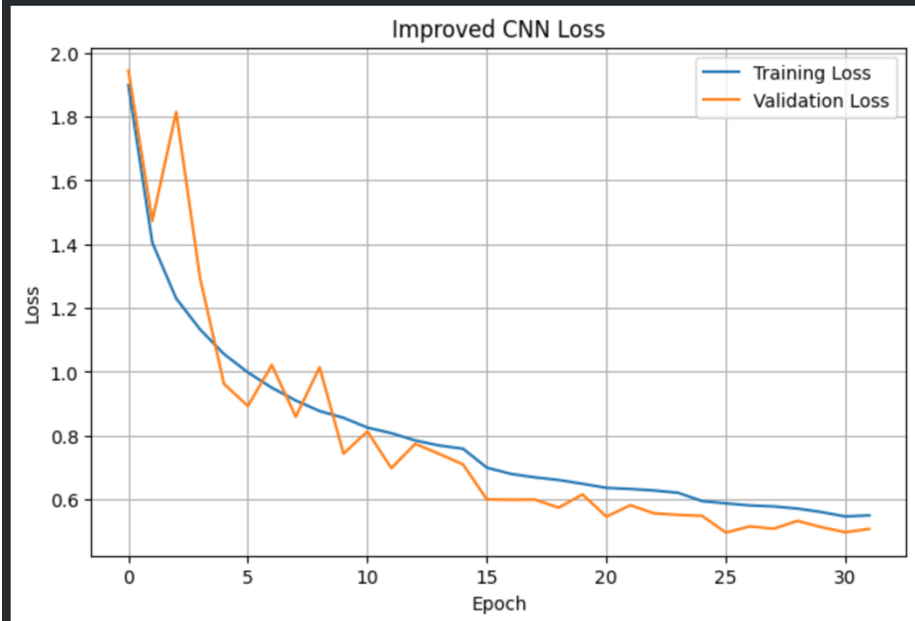
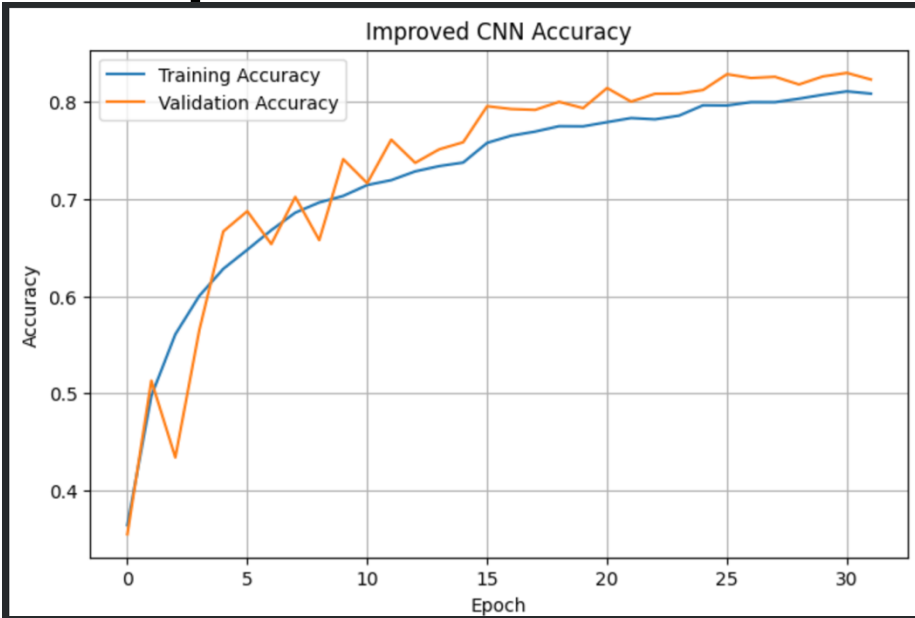
**Key takeaway:** The baseline CNN achieved 74.26% test accuracy, but the widening training-validation gap and rising validation loss indicate overfitting.

# Improved CNN Design and Training Strategy

| Data Augmentation      | Architecture                    | Training Config                         | Callbacks         |
|------------------------|---------------------------------|-----------------------------------------|-------------------|
| Random Horizontal flip | Deeper CNN than Baseline        | Optimiser : Adam                        | EarlyStopping     |
| Random Rotation (0.1)  | Batch normalisation             | Learning Rate: 0.001                    | ReduceLROnPlateau |
| Random Zoom (0.1)      | Stronger dropout regularisation | Batch size: 64 – Max Epochs 40          | —                 |
| —                      | Dense layer → 256 units         | Loss : Sparse categorical cross-entropy | —                 |
|                        | More robust feature learning    |                                         |                   |

**Goal:** Reduce overfitting observed in the baseline CNN and improve validation/test generalisation.

# Improved CNN - Results



| Metric                    | Result |
|---------------------------|--------|
| Final training accuracy   | 80.87% |
| Final validation accuracy | 82.33% |
| Best validation accuracy  | 83.01% |
| Test accuracy             | 82.51% |
| Final training loss       | 0.5499 |
| Final validation loss     | 0.5075 |
| Test loss                 | 0.5152 |

**Key takeaway:** The improved CNN reduced overfitting, increased test accuracy from **74.26%** to **82.51%**, and reduced test loss from **0.9621** to **0.5152**.

# Baseline vs Improved CNN Comparison

| Metric                    | Baseline CNN | Improved CNN | Change  |
|---------------------------|--------------|--------------|---------|
| Final training accuracy   | 88.74%       | 80.87%       | -7.87%  |
| Final validation accuracy | 73.72%       | 82.33%       | +8.61%  |
| Best validation accuracy  | 74.25%       | 83.01%       | +8.76%  |
| Test accuracy             | 74.26%       | 82.51%       | +8.25%  |
| Final training loss       | 0.3078       | 0.5499       | +0.2421 |
| Final validation loss     | 0.9557       | 0.5075       | -0.4482 |
| Test loss                 | 0.9621       | 0.5152       | -0.4469 |

**Key takeaway:** The improved CNN generalised better than the baseline, increasing test accuracy by **8.25 percentage points** and reducing test loss by **0.4469**, which indicates that augmentation and regularisation successfully reduced overfitting.

# Precision, Recall and F1-Score

|              | precision | recall | f1-score | support    |
|--------------|-----------|--------|----------|------------|
| airplane     | 0.841414  | 0.8330 | 0.837186 | 1000.0000  |
| automobile   | 0.913001  | 0.9340 | 0.923381 | 1000.0000  |
| bird         | 0.830823  | 0.7170 | 0.769726 | 1000.0000  |
| cat          | 0.787645  | 0.6120 | 0.688801 | 1000.0000  |
| deer         | 0.767161  | 0.8270 | 0.795958 | 1000.0000  |
| dog          | 0.802928  | 0.7130 | 0.755297 | 1000.0000  |
| frog         | 0.736675  | 0.9260 | 0.820558 | 1000.0000  |
| horse        | 0.858268  | 0.8720 | 0.865079 | 1000.0000  |
| ship         | 0.884016  | 0.9070 | 0.895360 | 1000.0000  |
| truck        | 0.841035  | 0.9100 | 0.874159 | 1000.0000  |
| accuracy     | 0.825100  | 0.8251 | 0.825100 | 0.8251     |
| macro avg    | 0.826297  | 0.8251 | 0.822551 | 10000.0000 |
| weighted avg | 0.826297  | 0.8251 | 0.822551 | 10000.0000 |

## Strongest Classes

| Class      | F1-score      |
|------------|---------------|
| Automobile | <b>92.34%</b> |
| Ship       | <b>89.54%</b> |
| Truck      | <b>87.42%</b> |
| Horse      | <b>86.51%</b> |

## Weakest Classes

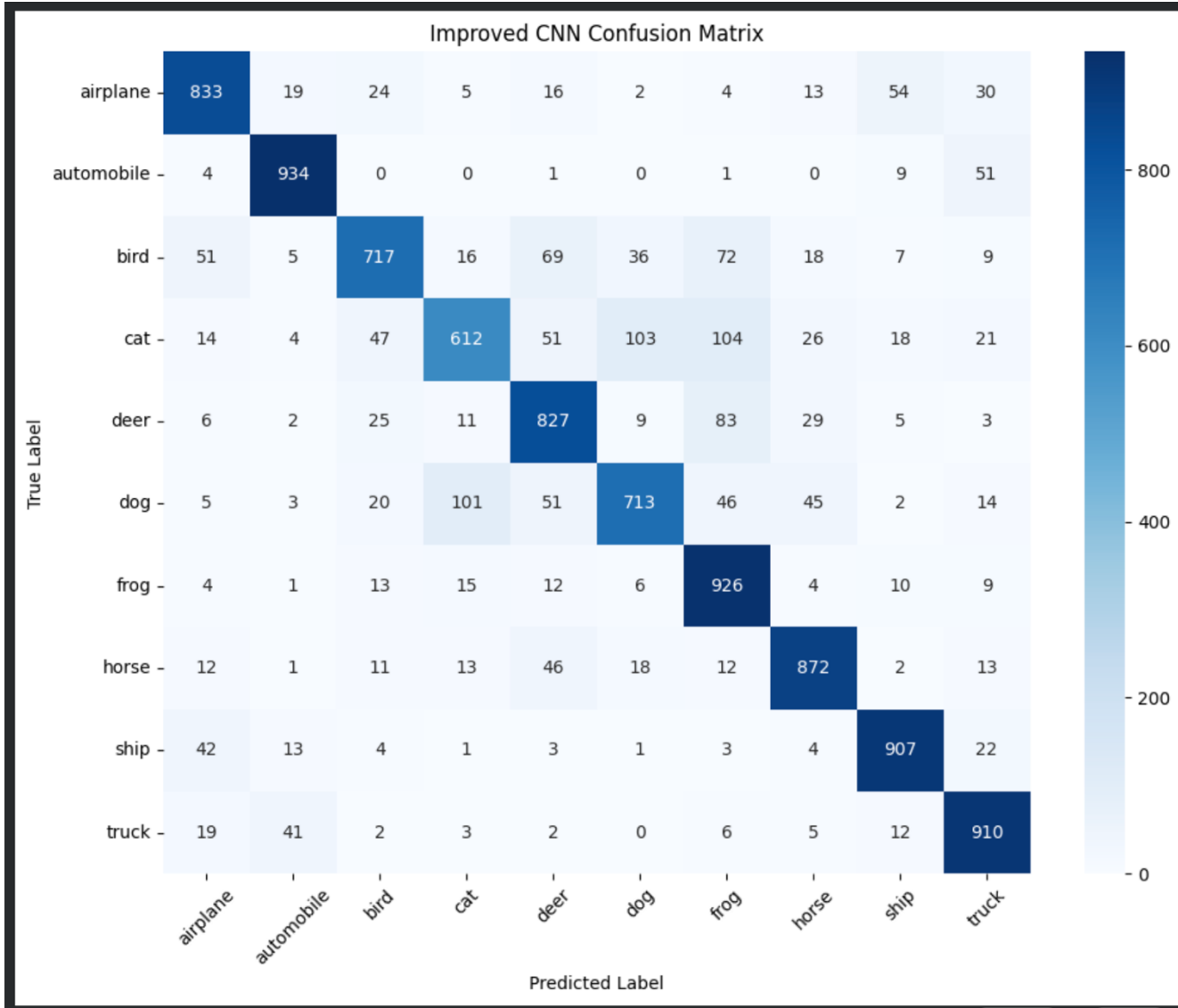
| Class | F1-score      |
|-------|---------------|
| Cat   | <b>68.88%</b> |
| Dog   | <b>75.53%</b> |
| Bird  | <b>76.97%</b> |

## Overall performance:

Accuracy = **82.51%**, Macro F1-score = **82.26%**,  
Weighted F1-score = **82.26%**

The classification report shows that the improved CNN achieved **82.51% overall accuracy**, with a **macro F1-score of 82.26%**. The strongest classes were **automobile, ship, truck, and horse**, while the weakest classes were **cat, dog, and bird**. This suggests that the model performs better on visually distinctive categories and struggles more with visually similar animal classes.

# Confusion Matrix



The confusion matrix confirms that the improved CNN performs strongly on visually distinctive classes such as automobile, ship, truck, frog, and horse. Most errors occur among visually similar animal classes, especially cat, dog, bird, deer, and frog. For example, cat images were often misclassified as frog or dog, while dog images were often misclassified as cat. This explains why animal classes had lower F1-scores in the classification report.

# Transfer Learning Consideration

| Area               | Key Point                                                                                |
|--------------------|------------------------------------------------------------------------------------------|
| What it is         | Reuses a pre-trained model such as MobileNetV2, ResNet50, or EfficientNetB0              |
| Why useful         | Can transfer visual features learned from large datasets                                 |
| CIFAR-10 challenge | Images are only 32×32, so resizing is required                                           |
| Trade-off          | Better feature reuse but higher compute and complexity                                   |
| Project decision   | Improved custom CNN used as main model; transfer learning proposed as future enhancement |

**Decision:** The improved custom CNN was used as the main implemented model because it achieved strong test performance at 82.51%. Transfer learning remains a suitable future enhancement, but would require image resizing and additional computational cost.

# Comparison of ML Approaches

| Track                             | Strengths                                                       | Limitations                                                 |
|-----------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------|
| Track 1: Classical ML             | Lower compute, easier to explain                                | Needs manual feature engineering; weaker for complex images |
| Track 2: CNNs / Transfer Learning | Learns spatial features automatically; strong image performance | Requires more compute, tuning, and regularisation           |
| Track 3: Self-Supervised / NAS    | Can reduce label dependency and automate architecture search    | More complex, harder to implement, higher compute cost      |

**Key takeaway:** Track 2 provided the best balance for this assignment: strong CIFAR-10 performance, manageable implementation complexity, and clear evaluation through accuracy, F1-score, and confusion matrix.



# Demo

```
100 #
# Choose a test image index.
# You can change this number to show a different example.
demo_index = 0

# Get image and true label
demo_image = X_test[demo_index]
true_label_index = y_test[demo_index]
true_label = class_names[true_label_index]

# Model expects a batch dimension (1, 32, 32, 3)
demo_image_batch = np.expand_dims(demo_image, axis=0)

# Generate prediction probabilities
demo_prediction_probs = improved_cnn.predict(demo_image_batch)

# Get predicted class and confidence
predicted_label_index = np.argmax(demo_prediction_probs[0])
predicted_label = class_names[predicted_label_index]
confidence = demo_prediction_probs[0][predicted_label_index]

# Display results
print("Actual label:", true_label)
print("Predicted label:", predicted_label)
print("Prediction confidence:", round(confidence * 100, 2), "%")

# Show top 5 probabilities
top_5_indices = np.argsort(demo_prediction_probs[0])[-5:][::-1]

print("Top 5 predicted classes:")
for i in top_5_indices:
    print(class_names[i], demo_prediction_probs[0][i] * 100, "%")

# -----
# Visual output for class/imag
# -----

plt.figure(figsize=(12, 5))

# Image display
plt.subplot(1, 2, 1)
plt.imshow(demo_image)
```

Variables | Terminal

10:34 PM | 14 Python 3

# Conclusion and Lessons Learned

|             |                                                                           |
|-------------|---------------------------------------------------------------------------|
| Main result | Improved CNN achieved 82.51% test accuracy                                |
| Lesson 1    | Baseline CNN learned well but overfit                                     |
| Lesson 2    | Augmentation and regularisation improved generalisation                   |
| Limitation  | Similar animal classes remained difficult                                 |
| Future work | Transfer learning, stronger augmentation, explainability such as Grad-CAM |

# References

- Krizhevsky, A., Nair, V. and Hinton, G. (2009) *The CIFAR-10 dataset*.
- TensorFlow (2024) *Convolutional Neural Network tutorial*.
- TensorFlow/Keras documentation for data augmentation, callbacks, and model training.
- He, K., Zhang, X., Ren, S. and Sun, J. (2015) *Deep Residual Learning for Image Recognition*.